

Track C Chapter — Run-Time Guided Token Pruning (RGTP) for Temporal Heads

This chapter documents Track C as a thesis-ready narrative and lab manual. It formalizes the motivation, theory, and full operational procedure for applying Run-Time Guided Token Pruning (RGTP) on top of a trained Track B head. The chapter is intentionally extensive (400+ lines) so it can be dropped into a thesis appendix or methods section without further expansion.

Table of Contents

- 0. Reading Guide and Scope
 - 1. Motivation, Problem Statement, and Assumptions
 - 2. Conceptual Link: Track A → Track B → Track C
 - 3. Data Assets, Paths, and Layout
 - 4. Inputs Required by Track C
 - 5. RGTP Theory — Importance Estimation and Pruning
 - 6. Interaction with Track B Components (Tokenizer, Fusion, Head)
 - 7. Configuration Files and Key Parameters
 - 8. RuntimeConfig and CLI Overrides
 - 9. Instrumentation (Latency, VRAM, FLOPs) — Theory and Practice
 - 10. Evaluation Pipeline (trackC_pruning.py) — Step-by-Step
 - 11. Metric Definitions and Interpretations
 - 12. Outputs and File Schemas
 - 13. Rate Sweeps and Pareto Curves
 - 14. Workflows and Recipes
 - 15. Troubleshooting and Failure Modes
 - 16. Reproducibility and Logging
 - 17. Integration with Track B and Track A Manifests
 - 18. Ablations and Sensitivity Analyses
 - 19. Security, Safety, and Cleanup
 - 20. Glossary
 - 21. Extended Notes and Commentary
 - 22. Example Commands (PowerShell/Linux)
 - 23. Example Metrics and Summaries
 - 24. Appendix A — Parameter Reference (RGTP, Evaluation, Instrumentation)
 - 25. Appendix B — File Map and Paths
 - 26. Appendix C — Pseudo-Code for RGTP Scoring and Pruning
 - 27. Appendix D — Latency/Throughput Calculation Details
 - 28. Appendix E — VideoMAE Variant Notes
 - 29. Appendix F — Checklists
-

0. Reading Guide and Scope

- Audience: thesis reviewers and practitioners who need both theoretical framing and executable steps.
- Style: prose plus operational bullets and code snippets; all paths relative to repository root.
- Scope: Track C only. It assumes a trained Track B checkpoint and Stage B manifests. Track A/B generation is referenced but not re-explained in depth.
- Goal: enable repeatable RGTP experiments, including latency/accuracy trade-off curves and semantic metric preservation.

1. Motivation, Problem Statement, and Assumptions

- Problem: Track B heads can be over-provisioned with candidate tokens; inference latency and VRAM can be reduced by pruning low-importance candidates while preserving accuracy and semantic metrics ($N/N_v/N + \delta/All$).
- RGTP approach: compute per-candidate importance using attention rollout plus motion cues, prune bottom fraction, and evaluate downstream metrics with instrumentation.
- Assumptions:
 - Track B checkpoint exists and matches the chosen backbone (ResNet18 default or VideoMAE ego).
 - Stage B val manifest available (from Track A Stage B run or manifests root).
 - Frames exist at `local_extraction/v2/extracted_frames`.
 - Pruning is evaluation-time only; no retraining.

2. Conceptual Link: Track A → Track B → Track C

- Track A Stage B: produces head manifests (`head_train/val.jsonl`) with candidate boxes and semantics.
- Track B: trains temporal head (fusion + multi-task heads) on those manifests; outputs checkpoints and metrics.
- Track C: loads Track B checkpoint, applies RGTP during inference, and measures accuracy/latency trade-offs. No new learning occurs.
- Output of Track C: metrics JSONs and optional plots comparing pruned vs unpruned performance.

3. Data Assets, Paths, and Layout

- Frames: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`
- Stage B manifests (preferred):
`local_extraction/runs/Track_A/trackA_stageB_<timestamp>/head_val.jsonl` (or `.json`)
- Track B checkpoints: `local_extraction/runs/Track_B/checkpoints/trackB_best.pt` (or `trackB_final_*.pt`)
- Track C outputs: `local_extraction/runs/Track_C/{metrics,plots}/`
- Configs: `local_extraction/configs/trackC.yaml` (inherits `base.yaml`)
- Scripts: `trackC_pruning.py` , `trackC_compare_metrics.py` , `trackC_plots.py`

4. Inputs Required by Track C

- Checkpoint: Track B checkpoint with projector/fusion/head weights and optional noun/verb/bin heads.

- Val manifest: Stage B val manifest; auto-discovery picks latest Stage B run if not specified.
- Frames: present under `paths.extracted_frames` (default `local_extraction/v2/extracted_frames`).
- Optional: hotspot/CLIP are not used in Track C; pruning acts on tokens before the head.

5. RGTP Theory — Importance Estimation and Pruning

- Goal: reduce inference cost by discarding low-importance candidate tokens before head forward.
- Importance score per spatial token combines:
 - **Rollout**: FGTP attention from frame t-1 (saliency from fusion).
 - **Motion**: energy of token change from t-1 to t ($(x_t - x_{t-1})^2$ mean).
 - **Blend**: `score = temporal_decay * rollout + (1 - temporal_decay) * motion`.
- Candidate score: ROI-pool the blended grid over each candidate box.
- Pruning rule:
 - Keep `K = max(min_keep, round(N * (1 - rate)))` candidates by score.
 - Dropped candidates filled with `LOGIT_FILL` (large negative logit) so their probability is near zero; TTC and semantics logits filled similarly.
- Safety: `min_keep` ensures at least a floor of candidates remain to avoid empty predictions.
- Typical rates: 0.0 (baseline), 0.1, 0.3, 0.5, 0.7.

6. Interaction with Track B Components (Tokenizer, Fusion, Head)

- Track C reuses Track B tokenizer, fusion, and head without modification.
- TokenizerConfig selects backbone (ResNet18 or VideoMAE). CLI `--video_backbone` can override.
- Fusion and head are rebuilt using checkpoint weights; class counts and optional heads (noun/verb/bin) inferred from weight shapes.
- Projector input dim inferred from config/backbone (512 for ResNet18, 768 for VideoMAE ego).
- ROI pooling uses `roi_pool_tokens_mean` on fused image/video tokens for each candidate.

7. Configuration Files and Key Parameters

- File: `local_extraction/configs/trackC.yaml`
- RGTP:
 - `rgtp.enabled` : toggle pruning.
 - `rgtp.rate` : fraction to drop (0.0–0.95).
 - `rgtp.min_keep` : floor on kept candidates.
 - `rgtp.temporal_decay` : rollout vs motion blend.
 - `rgtp.logit_fill` : negative logit for pruned tokens.
- Evaluation:
 - `evaluation.checkpoint` : explicit path or `null` to auto-detect.
 - `evaluation.stageB_run` , `evaluation.val_manifest` : `null` for auto-discovery.
 - `evaluation.batch_size` : default 1 (per-frame pruning).
 - `evaluation.candidate_limit` : cap before pruning.
 - `evaluation.normalize_ttc` : use normalized TTC from dataset.
- Instrumentation:

- `instrumentation.record_latency/vram/flops` , `use_cuda_events` , `warmup_iters` , `bench_iters` , `bench_samples` .
 - Rate sweep:
 - `rate_sweep.enabled` , `rate_sweep.rates` for scripted sweeps (not implemented in main script by default).
-

8. RuntimeConfig and CLI Overrides

- RuntimeConfig (in-code defaults; see `trackC_pruning.py`):
 - Paths: `checkpoint` , `stageB_run` , `val_manifest` .
 - Pruning: `pruning_enabled` , `rgtp_rate` , `min_keep` .
 - Instrumentation: `measure_latency` , `measure_vram` , `measure_flops` , `bench_warmup` , `bench_iters` , `bench_samples` .
 - CLI arguments:
 - `--config trackC` (default)
 - `--video_backbone {resnet18,videomae_ego}`
 - `--rgtp_rate <float>`
 - `--checkpoint <path>`
 - `--no_pruning` (sets rate=0)
 - CLI overrides take precedence over YAML and RuntimeConfig defaults.
-

9. Instrumentation (Latency, VRAM, FLOPs) — Theory and Practice

- Latency:
 - CUDA events when available; falls back to wall-clock.
 - Metrics: mean/median/p90/p95 latency; throughput (samples/s, candidates/s).
 - VRAM:
 - Peak allocated and reserved (CUDA only); zero on CPU.
 - FLOPs:
 - Optional one-off profile of head forward pass; may be expensive.
 - Micro-benchmark:
 - `bench_warmup` warmup iterations; `bench_iters` timed iterations; `bench_samples` number of samples benchmarked.
 - Use cases:
 - Compare latency vs pruning rate to build Pareto curves.
 - Verify VRAM savings when pruning aggressively.
-

10. Evaluation Pipeline (`trackC_pruning.py`) — Step-by-Step

1. Parse CLI; load YAML config; pick backbone.
2. Resolve Stage B run (latest Track A Stage B) and val manifest (head_val.*).
3. Resolve Track B checkpoint (`checkpoint` override or latest/final).
4. Build TokenizerConfig; build TrackBDataset (val) with candidate_limit and TTC normalization.
5. Load models: projector, fusion, head; infer class counts and optional heads; move to device.
6. Iterate samples (DataLoader workers=0):

- Project img/vid tokens; fuse via Track B fusion.
 - Compute rollout and motion; blend to grid_score.
 - ROI-pool grid_score to candidate scores; build keep mask.
 - Run head on kept candidates; fill dropped slots with `LOGIT_FILL` /zeros.
 - Collect logits, TTC (denorm), labels, semantics; update frame-level metrics.
 - Record latency/benchmark/FLOPs/VRAM if enabled.
7. Aggregate metrics; compute mean prune fraction; add runtime stats.
 8. Save metrics JSON and summary JSON under `runs/Track_C/metrics/`.
 9. Print key metrics; log via RunLogger if available.
-

11. Metric Definitions and Interpretations

- Candidate-level:
 - `accuracy` : next-active accuracy (binary or multi-class).
 - `mAP` : binary AP for class 1 or macro for multi-class.
 - `ttc_mae_seconds` : mean absolute error on denormalized TTC.
 - Pruning stats:
 - `rgtp_enabled` , `rgtp_rate_request` , `rgtp_mean_fraction_pruned` .
 - `samples_total` , `samples_without_candidates` .
 - Frame-level (top-1):
 - `N_mAP` : noun correct + IoU $\geq$ 0.5 on top candidate.
 - `Nv_mAP` : noun+verb correct + IoU $\geq$ 0.5.
 - `N_delta_mAP` : noun + TTC-bin correct + IoU $\geq$ 0.5.
 - `All_mAP` : noun+verb+TTC-bin correct + IoU $\geq$ 0.5.
 - Frame-level (top-5 hit):
 - `N_top5_acc` , `Nv_top5_acc` , `N_delta_top5_acc` , `All_top5_acc` .
 - Frame-level (top-5 AP):
 - `N_top5_mAP` , `Nv_top5_mAP` , `N_delta_top5_mAP` , `All_top5_mAP` .
 - Per-class breakdown:
 - `per_noun_stats` , `per_verb_stats` with total/correct/accuracy per ID.
 - Runtime:
 - Latency quantiles, throughput, head benchmark mean, peak VRAM, FLOPs (when available).
-

12. Outputs and File Schemas

- Metrics: `local_extraction/runs/Track_C/metrics/trackC_val_rateXX_<timestamp>.json`
 - Contains metrics, pruning stats, runtime stats, checkpoint/manifest paths.
- Summary: `trackC_val_rateXX_<timestamp>_summary.json`
 - Wraps metrics + serialized eval/rgtp/instrumentation configs.
- Plots (via `trackC_plots.py`):
 - `plots/trackC_metrics_over_time.png`
 - `plots/trackC_metrics_summary.tsv`
- Example metrics JSON excerpt:

```
{
  "accuracy": 0.59,
  "mAP": 0.33,
  "ttc_mae_seconds": 0.19,
  "rgtp_mean_fraction_pruned": 0.50,
  "latency_ms_mean": 12.5
}
```

13. Rate Sweeps and Pareto Curves

- Purpose: quantify accuracy-vs-latency trade-offs by varying `rgtp.rate`.
- Manual sweep: edit `rgtp.rate` or use `--rgtp_rate` for each run; collect metrics; plot with `trackC_plots.py`.
- Suggested rates: `[0.0, 0.1, 0.2, 0.3, 0.4, 0.5]` for coarse; include 0.7 for aggressive pruning.
- Plot axes:
 - x: `latency_ms_mean` or `throughput_samples_per_s`.
 - y: `mAP` (or `Nv_mAP`/`All_mAP` for semantic preservation).
 - Secondary: VRAM or FLOPs if instrumented.

14. Workflows and Recipes

- **Baseline vs Pruned Comparison:**
 - Run with `--no_pruning` (rate=0) to capture baseline metrics.
 - Run with `--rgtp_rate 0.3` (or chosen rate).
 - Compare metrics JSON via `trackC_compare_metrics.py` or manual diff.
- **Latency-Focused Run:**
 - Enable instrumentation; keep `candidate_limit` modest (e.g., 16).
 - Monitor `latency_ms_*` and throughput; choose rate that meets budget.
- **Semantic Integrity Check:**
 - Ensure Track B checkpoint has noun/verb/bin heads.
 - Inspect `Nv_mAP` and `All_mAP` across pruning rates.
- **VideoMAE Backbone Evaluation:**
 - `--video_backbone videomae_ego`; ensure checkpoint matches backbone; adjust batch size if needed.

15. Troubleshooting and Failure Modes

- **No val manifest found:** set `RuntimeConfig.val_manifest` explicitly; ensure Stage B run exists.
- **Checkpoint missing:** set `RuntimeConfig.checkpoint` to a valid Track B checkpoint; train Track B if absent.
- **CUDA OOM:** reduce `candidate_limit`; disable benchmarking; lower `rgtp_rate` to prune more; fall back to CPU (slower).
- **Latency zeros or missing:** check `measure_latency`; CUDA events need GPU; CPU uses wall-clock.

- **VRAM metrics zero:** expected on CPU; disable `measure_vram` to reduce noise.
 - **FLOPs missing:** profiling may fail on some builds; disable `measure_flops` if unstable.
 - **Mismatch noun/verb IDs:** checkpoint vocab inferred from training data; ensure val manifest uses same taxonomy IDs.
 - **Empty candidates skipped:** manifests with zero boxes are skipped; counts reported in `samples_without_candidates`.
 - **Backbone mismatch:** ResNet18 checkpoints will not load VideoMAE tokens; match checkpoint to `video_backbone`.
 - **Prune mask keeps all:** if `rate` near 0 or `min_keep` $\geq$ num candidates, pruning has no effect; adjust rate/min_keep.
-

16. Reproducibility and Logging

- Config provenance: summary JSON records eval/rgtp/instrumentation configs; metrics include checkpoint/manifest paths.
 - RunLogger: Track C attempts to log configs, metrics, artifacts; warnings are non-fatal if logger missing.
 - Naming: metrics filename encodes rate and timestamp; keep a short text note per run for thesis tables.
 - Checkpoint provenance: store which Track B checkpoint was used (best vs final).
-

17. Integration with Track B and Track A Manifests

- Track B checkpoint: must match backbone and include head weights; class counts inferred from weights.
 - Stage B manifests: preferred source is latest Stage B run; fallback to `local_extraction/v2/manifests`.
 - Candidate_limit: applied before pruning; set consistently with Track B eval for fair comparisons.
 - TTC mode: if bin head exists, ttc_mode defaults to bin; otherwise regression with bin-from-reg logic for N_delta/All metrics.
-

18. Ablations and Sensitivity Analyses

- Pruning rate sweep: observe mAP, Nv_mAP, All_mAP vs latency.
 - Temporal_decay sweep: test `{0.3, 0.6, 0.8}` to balance rollout vs motion.
 - Min_keep sweep: `{1,2,4}` to avoid over-pruning small candidate sets.
 - Candidate_limit sweep: `{8,16,32}` to see upstream effect on pruning efficacy.
 - Backbone comparison: ResNet18 vs VideoMAE ego; note projector in-dim and token shapes.
 - Instrumentation on/off: measure overhead of instrumentation itself; run once with instrumentation disabled for clean latency.
-

19. Security, Safety, and Cleanup

- No network calls; all operations local.
- Runs are timestamped; metrics files do not overwrite existing ones.
- Cleanup: delete old `runs/Track_C/metrics` and `runs/Track_C/plots` to save disk.
- Avoid editing manifests in-place; copy if manual fixes are needed.

20. Glossary

- **RGTP**: Run-time Guided Token Pruning (rollout + motion-based importance).
 - **LOGIT_FILL**: large negative logit assigned to pruned candidates.
 - **FGTP**: Frame-Guided Temporal Pooling (Track B fusion).
 - **N/Nv/N+ δ /All**: frame-level semantic metrics using noun, noun+verb, noun+TTC-bin, noun+verb+TTC-bin.
 - **TTC**: Time-to-contact (seconds).
 - **Rate**: fraction of candidates dropped; mean fraction pruned reported post hoc.
 - **Instrumentation**: latency/VRAM/FLOPs measurement toggles.
-

21. Extended Notes and Commentary

- RGTP is training-free: importance derived from existing fusion tokens; no gradient updates.
 - Pruning acts on candidate dimension, not on spatial grid tokens directly; it removes ROI features before head.
 - LOGIT_FILL is chosen to push pruned candidates to near-zero probability; adjust only if you observe numerical saturation.
 - Pruning interacts with candidate_limit: if limit is already low, high pruning rates can collapse predictions; monitor min_keep.
 - For thesis reporting, include both quality and runtime metrics; show Pareto frontier across rates.
-

22. Example Commands (PowerShell/Linux)

- Default run (auto-discover checkpoint/manifest, pruning enabled):

```
python local_extraction\trackC\trackC_pruning.py
```

- Baseline (no pruning):

```
python local_extraction\trackC\trackC_pruning.py --no_pruning
```

- Custom pruning rate and checkpoint:

```
python local_extraction\trackC\trackC_pruning.py --rgtp_rate 0.3 --  
checkpoint local_extraction/runs/Track_B/checkpoints/trackB_best.pt
```

- VideoMAE backbone:

```
python local_extraction\trackC\trackC_pruning.py --video_backbone
videomae_ego --checkpoint
local_extraction/runs/Track_B/checkpoints/trackB_vmae_best.pt
```

- Plot metrics:

```
python local_extraction\trackC\trackC_plots.py --runs_dir
local_extraction/runs/Track_C --out_dir
local_extraction/runs/Track_C/plots
```

- Compare Track B vs Track C:

```
python local_extraction\trackC\trackC_compare_metrics.py
```

23. Example Metrics and Summaries

- Minimal metrics JSON structure:

```
{
  "accuracy": 0.65,
  "mAP": 0.40,
  "ttc_mae_seconds": 0.25,
  "rgtp_mean_fraction_pruned": 0.30,
  "latency_ms_mean": 9.8,
  "throughput_samples_per_s": 102.0,
  "N_mAP": 0.33,
  "Nv_mAP": 0.18,
  "N_delta_mAP": 0.29,
  "All_mAP": 0.16
}
```

- Summary JSON adds serialized configs (`eval_config` , `rgtp_config` , `instrumentation`).
- Plots TSV contains columns discovered in metrics files (including runtime fields if present).

24. Appendix A — Parameter Reference (RGTP, Evaluation, Instrumentation)

- RGTP:
 - `enabled` : True/False
 - `rate` : float [0,0.95]
 - `min_keep` : int ≥ 1
 - `temporal_decay` : float [0,1]
 - `logit_fill` : negative float
 - Evaluation:
 - `checkpoint` , `stageB_run` , `val_manifest`
 - `batch_size` : default 1
 - `candidate_limit` : e.g., 16
 - `normalize_ttc` : True/False
 - `num_workers` : keep 0
 - Instrumentation:
 - `record_latency` , `record_vram` , `record_flops` , `use_cuda_events`
 - `warmup_iters` , `bench_iters` , `bench_samples`
 - Rate sweep:
 - `enabled` , `rates` list
-

25. Appendix B — File Map and Paths

- Scripts:
 - `trackC_pruning.py` — main RGTP evaluation.
 - `trackC_compare_metrics.py` — compare Track B vs Track C metrics.
 - `trackC_plots.py` — plot Track C metrics (including runtime).
 - Configs:
 - `configs/trackC.yaml` — main settings (inherits `base.yaml`).
 - Outputs:
 - `runs/Track_C/metrics/trackC_val_rateXX<ts>.json` and `_summary.json`
 - `runs/Track_C/plots/trackC_metrics_over_time.png` , `trackC_metrics_summary.tsv`
 - Inputs:
 - Track B checkpoints: `runs/Track_B/checkpoints/`
 - Stage B manifests: latest `runs/Track_A/trackA_stageB_*` or `local_extraction/v2/manifests/`
 - Frames: `local_extraction/v2/extracted_frames/`
-

26. Appendix C — Pseudo-Code for RGTP Scoring and Pruning

```
# given fused tokens and candidate boxes
rollout = rollout_attention(fusion, img_tokens, vid_tokens)      # (N,)
motion  = motion_energy(vid_tokens)                             # (N,)
grid    = decay * rollout + (1 - decay) * motion
scores  = [roi_pool(grid, box) for box in boxes]
keep    = max(min_keep, round(len(scores) * (1 - rate)))
mask    = topk_mask(scores, keep)
```

```
out      = head(on kept candidates)
fill pruned slots with LOGIT_FILL (and zero TTC) to preserve shape
```

27. Appendix D — Latency/Throughput Calculation Details

- Latency lists collect per-sample durations (ms).
- Percentiles computed from sorted list: p90, p95.
- Throughput:
 - `throughput_samples_per_s = num_samples / total_time_seconds`
 - `throughput_candidates_per_s = total_candidates / total_time_seconds`
- Micro-benchmark uses `torch.utils.benchmark` ; warmup discarded; mean reported.

28. Appendix E — VideoMAE Variant Notes

- Backbone override via `--video_backbone videomae_ego` or config.
- Projector in-dim set to 768; ensure checkpoint matches.
- Tokens: if using pre-extracted tokens in Track B training, the checkpoint still stores projector weights; Track C rebuilds projector accordingly.
- Memory: VideoMAE tokens larger; consider pruning to maintain latency benefits.

29. Appendix F — Checklists

- **Pre-run:**
 - ☐ Track B checkpoint path valid.
 - ☐ Val manifest path resolves (Stage B run present).
 - ☐ Frames accessible under `frames_root`.
 - ☐ Pruning rate and `min_keep` set.
 - ☐ Instrumentation toggles as desired.
- **Post-run:**
 - ☐ Metrics JSON written under `runs/Track_C/metrics` .
 - ☐ Summary JSON captured.
 - ☐ (Optional) Plots generated.
 - ☐ Compare with baseline (no pruning) for thesis tables.

This chapter is intentionally verbose to serve as a single, thesis-ready reference for Track C, covering both theoretical rationale for RGTP and the exact steps to reproduce pruning-versus-latency experiments locally.